

Figure 3: Igelle's architecture



Figure 4: The Esther-panel application resembles Apple's Macintosh quick-launch

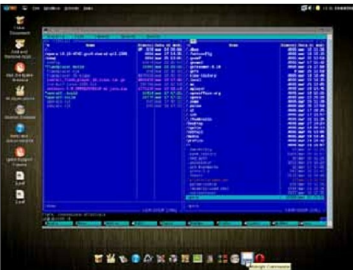


Figure 5: Midnight Commander compiled and deployed within Igelle DSV

```
# mkdir /usr/lib/locale
# localedef -ci fr_FR -f UTF-8 fr
```

When you launch the calculator application with the `LANG` environment variable set to “fr” (`LANG=fr calculator`), you’ll see its menus in French. However, to make the full desktop appear in French, by default, you should add the following line into `~/config/esther-session.config`:

```
esther-session-control setenv LANG fr
```

All GNOME applications used by the system expect to have localisation files for each language. If you correctly configured your language, but an application is still displayed

in English, then this means that your particular application has no translation file yet; this job still needs to be done by the translation team.

What about input in a local language? That’s easy—place the following command into a start-up script (for example, `.bashrc` in your home directory), and you can now switch between US and Arabic keyboards by pressing both `Shift` keys:

```
setxkbmap -option grp:switch,grp:shift_toggle,grp_led:scroll us,ar
```

Self-contained packages

What is a self-contained package, and why is it needed? Imagine that you have a package called *MyEditor*. It keeps all files (localisation files, dynamic libraries, icons and so forth) in one specific location—say, `/opt/MyEditor`. Is it comfortable? Very! Besides, you have an opportunity to install and uninstall this package from the system in the blink of an eye, just like the `mount/umount` commands, but in this case, using the `sjdctl` command.

By default, the Igelle system has bleeding-edge software only, like the latest X11 libraries (and those that developers require), GNOME and QT, and the GCC compiler—that’s why when you run Igelle, you can compile any application from source. For example, in Live CD mode, I was able to compile *MPlayer*, *rdesktop* and *vncviewer*, from scratch. This definitely helped me a lot when I faced an emergency situation. To view a full list of packages that are pre-installed in the *rootfs*, use: *Igelle* > *About this system* > *Packages*.

The CD image’s contents are limited by size (650 MB) and unfortunately, some much-required applications like Kino, AbiWord or VLC weren’t included. However, you can find them in a special repository [3]. All you need is to download files with the *sjapp* extension, and install it. Alternatively, you can download a specification [4] file only (with an *sj*s extension), which describes how the package should be built, and create a binary package by yourself.

A specific package that can’t be found even in this repo has to be built manually. In one particular case, I preferred using a file manager like Midnight Commander. I decided to compile the package for it first, and place it in the `/opt/mc` directory (exactly where an *mc* binary package would be installed). In order to compile the binary package, you might need to be familiar with the meta-compiler syntax, and how to correctly edit a package description. Let’s look at the `mc-4.7.1.sjs` file:

```
mc: sjapp {
    setself "url" "http://www.midnight-commander.org/downloads/35"
    setself "version" "4.7.1"
```